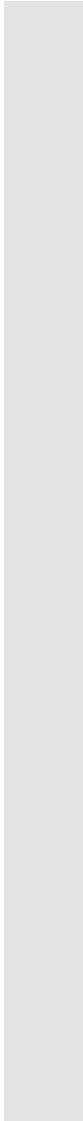




Object Oriented Programming



DATATYPES
VARIABLES and CONSTANTS
OPERATORS
EXPRESSION
COMMENTS

MARVIN C. SANTOS, MSIT

Instructor

DATATYPES

Types of DATA

- 1. Numbers
- 2. Text
- 3. Images / Photo
- 4. Video
- 5. Logical Data

C#

Data Types
Categories

- **1. Value Type**
 - Numeric Types
 - Boolean Types
 - Characters
 - Enumerations
- *2. Reference Type*

C#

Value Types

- **1. Value Type**

- **Numeric Types**

- int** = 32-bit signed integers

- short** = 16-bit signed integers

- long** = 64-bit signed integers

- byte** = 8-bit unsigned integer

- float** = single-precision floating-point numbers

- double** = double-precision floating-point numbers

- decimal** = decimal numbers with high precision

- **Boolean Types** = either true or false

- **Characters** = single Unicode character

- **Enumerations** = set of named constant values

float, double, decimal

C#

float = **single-precision** floating-point type, which means it has 32 bits to represent a number.

NOT suitable for situations where high precision is required

Example: **float pi = 3.1415926f;**

double = **double-precision** floating-point type, which means it has 64 bits to represent a number.

Larger range and higher precision than

Example: **double pi = 3.14159265358979;**

decimal = **high-precision** numeric type with 128 bits of storage.

Provides much higher precision and is ideal for financial and monetary calculations where accuracy is crucial.

Example: **decimal price = 19.99m;**

float = when you need to store real numbers with limited precision, and you want to save memory

double = most general-purpose floating-point calculations

decimal = when you need high precision, especially for financial calculations

	int	-2147483648 to 2147483647
	char	A
	byte	0 to 255
	decimal	decimal
	double	1.7e-308 to 1.7e+308
	float	3.4e-038 to 3.4e+038
	long	-9223372036854775808 to 92233720368547758058
	sbyte	
	uint	-128 to +127
	ulong	0 to 4,294,967,295
	bool	0 to 18,446,744,073,709,551,615 True or False

VARIABLES

What is Variable?

- Used to store and manipulate data.
- *Essential for writing programs :*
 - Allow you to work with values
 - Perform calculations
 - Store information

Aspects

Aspects of variables in C#:

Declaration: must be declared before they can be used

```
int age;
```

Initialization: giving it an initial value

```
int age = 30;
```

Data Types: various data types for different kinds of values

```
int age = 30;  
float price = 19.99f;  
char grade = 'A';
```

Scope: determines where in the code they can be accessed

Lifetime: duration during which it exists in memory

```
int age = 20;  
0 references  
void myMethod()  
{  
    int age = 25;  
    // ...  
}
```

Aspects

Aspects of variables in C#:

Mutability: mutable or immutable

```
const int interestRate = 42;
```

Naming Conventions: naming conventions for variables

`age`

`isStudent`

Type Inference: automatically determined by the compiler based on the assigned value.

```
var age = 30;
```

Nullability: nullable reference types

```
string? nullableString = null;
```

Value Assignment: can be assigned new values

```
int score = 90;  
score = 95;
```

Variable AS "Containers"

- Container for storing Data Values
 - Storage
 - Labeling
 - Updating Contents
 - Data Retrieval
 - Data Manipulation
 - Data Scope

Variable AS "Containers"

- Storage

```
int age = 30;
```

- Labeling

```
int age = 30;
```

- Updating Contents

```
int score = 90;  
score = 95;
```

- Data Retrieval

```
int age = 30;  
Console.WriteLine("Your age: " + age);
```

- Data Manipulation

```
int questions = 7;  
int essay = 2;  
int totalQuizes = questions + essay;
```

- Data Scope

```
int age = 20;  
0 references  
void myMethod()  
{  
    int age = 25;  
    // ...  
}
```

Variable Samples

- **In C#...**
 - int
 - double
 - char
 - string
 - bool

Variable

Syntax:

datatype variableName = value;

Example:

int MyAge = 20;

bool isActive = true;

float subjectGrade = 74.5f;

CONSTANT

DATA TYPES:

Different Constant

1. Integer constant

0, 1, 2, 123, 5767, 05, 0X23, 0xFFFF

2. Float constant

0.5, 35.05, 2.3e6, 3.52f or 3.52F, PI = 3.14

3. Character Constants

'g', 'D', ' ', '#'

4. String Constant

Hello Friends", "Computer", "5987", " "

Samples Constant

```
const int MaxValue = 100; //  
const double Pi = 3.14159265359;  
const string Greeting = "Hello, World!";  
  
int radius = 5;  
double circleArea = Pi * radius * radius;
```

CONSTANT offers Readability and Maintainability benefits by giving meaningful names to fixed values used throughout your code.

VARIABLES and CONSTANT

Variables and Constant



Variables and Constant

- **VARIABLE:** A name given to a storage location in memory.
 - A data item whose value can be change during the program's execution
 - *"Value can Vary"*
- **CONSTANT:** An immutable value
 - A data item whose value cannot change during a program execution
 - *"Value is Constant"*

Variables and Constant

VARIABLE = Hold Values

(which can be assigned and changed)

CONSTANT= Fix Values

*"The choice between using a variable or a constant **depends** on whether you expect the value to change or remain the same throughout your program"*

DEFINING a VARIABLES

RULES:

Defining Variable Name

- 1.The variables are case sensitive.
- 2.The variable name must start with an alphabet letter or underscore.
- 3.It includes the letter, digits and underscore.
- 4.There should not be a white space in a variable name.
- 5.The name of a variable should not be any reserved keywords like int, float, str, char, etc.
- 6.Use meaningful names.

WHY?

Follow these rules for
naming variables

*"Help you write clean and maintainable C#
code that is easy to understand and
collaborate on with others"*

Naming Conventions

1. Camel Case

totalCount myAge myVariable

2. Pascal Case

TotalCount MyAge MyVariable

3. Hungarian Notation

strFirstName intAge

OPERATORS and EXPRESSION

Operators

- Is a system that tells the compiler to perform specific task
- Used in mathematical or Logical Manipulations

Operators in C#

Assignment Operator

Arithmetic Operator

Relational Operators

Logical Operators

Bitwise Operators

Misc Operators

Operators in C#

Assignment Operator

Assignment Operators

- Used to **ASSIGN** values to variables
- Used **ALSO** to **UPDATE** the value of a variable
- Allow us to **Store values** in variables for later use or manipulations

Assignment Operators

Assignment Operator		
Assignment	=	a = 1
Addition Assignment	+=	a += 3
Subtraction Assignment	-=	a -= 3
Multiplication Assignment	*=	a *= 3
Division Assignment	/=	a /= 3

Assignment Operators

Assignment Operator

=

(Equal Operator)

Fundamental operator

```
int age = 30;
```

Assignment Operators

= VS ==

= used to store values in variable

== used to testing equality

```
int a = 10;  
int b = 20;  
bool isAxis = (a == b);  
Console.WriteLine(isAxis);
```

Operators in C#

Arithmetic Operator

Arithmetic Operators

- Used to perform basic **MATHEMATICAL OPERATIONS** on numeric values.

Commonly used arithmetic operators:

- Addition
- Subtraction
- Multiplication
- Division
- Integer Division
- Modulus
- Exponentiation

Arithmetic Operators	Description
+	Adds two operands
-	Subtracts second operand from the list
*	Multiplies both operands
/	Divides numerator by de-numerator
%	Modulus Operator and remainder of after an integer division
++	Increment operator
--	Decrement operator

**Operators in
C#**

Relational Operators

Relational Operators

- Used to **COMPARE** two values or expressions and return a *boolean result*.
- Allow us to compare values.
- Often used within conditional statements to create “Decision-Making structures”

Relational Operators

	Relational Operator
==	A == B
!=	A != B
>	(A>B)
<	(A<B)
>=	A >= B
<=	A <= B

Operators in C#

Logical Operators

Logical Operators

- Used to **PERFORM** logical operations on Boolean values or expressions.
- Allow us to combine or modify conditions in complex ways.

Logical Operators

	Logical Operator
&&	Returns TRUE if BOTH operands are 'true'
	Returns TRUE if at least 1 operands is 'true'
!	Unary operator that returns the OPPOSITE Boolean value of its operand

► Results ALWAYS True or False

Logical Operators

Let's Try:

```
x = True
```

```
y = False
```

```
result1 = x and y
```

False

```
result2 = x or y
```

True

```
result3 = not x
```

False

Logical Operators

- Used to **PERFORM** logical operations on Boolean values or expressions.
- Allow us to combine or modify conditions in complex ways.

Using AND (&&)

Operand	Operator	Operand	RESULT
True	&&	True	True
True	&&	False	False
False	&&	True	False
False	&&	False	False

Using OR (||)

Operand	Operator	Operand	RESULT
True		True	True
True		False	True
False		True	True
False		False	False

Using NOT (!)

Operand	Operator
!False	True
!True	False

EXPRESSIONS

Expression

- In C# : combination of operands
- (variables, literals, method calls)
- Operators that can be evaluated to single value

Expression

```
hasVisa = false;  
hasPassport = true;  
bool isValidTourist = (age >= 18) && (hasVisa || hasPassport);  
Console.WriteLine(isValidTourist);
```

COMMENTS

Single-Line

```
// Declaring my variable here  
int a = 1;
```

Multi-Line

```
• /*
```

Here's the code for you to revise
updated sept14 2022

```
*/
```

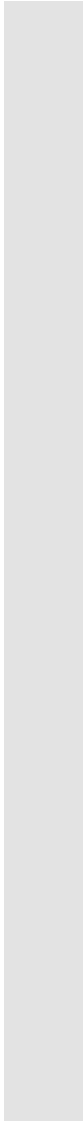
```
int a = 1;
```

```
int a = 10;  
int b = 20;  
bool isAxis = (a == b);    // Check if 'a' is equal to 'b' and store the result in 'isEqual'.  
Console.WriteLine(isAxis);
```

Comments

Comments are:

- Essential for improving code readability
- For documenting the purpose
- Usage of various parts of your code
- Properly documented code is easier to understand and maintain
- Especially when working on larger projects or collaborating with other developers.



Thank You